PRODUCTION GUIDE

VaultX Backend Engineering

Gemini AI / LLM Backend Integration Guide

Node.js Express me Google Gemini 2.5 Flash SDK, Structured JSON, AI Financial Advisor & Redis Caching Layer Lagane Ka Complete Manual

Kyun Zaroori Hai Yeh Guide?
Normal freshers AI ke naam par sirf ChatGPT wrapper banate hain. Real-world backend me AI ko **Database Context (RAG)**, **Strict Structured JSON Output**, aur **Redis Caching Layer** ke sath integrate kiya jata hai taaki sub-millisecond responses aayein aur API rate limits exhaust na hon.

Part 1: 100% Free Gemini API Key Setup (Bina Credit Card Ke)

Google AI Studio developers ko **Gemini 2.5 Flash** aur **Gemini 2.0 Flash** ka fully functional **FREE TIER** deta hai (15 requests/min, daily 1500 calls). Lekin 90% log ek choti galti karte hain jis se `403 PERMISSION_DENIED` error aata hai.

⚠️ **Sabse Badi Galti (403 Error se kaise bachein):**
Agar key banate waqt kisi **Purane Google Cloud Project** (jaise OAuth wala project) ko select kar liya, toh Google use Paid Cloud Project samajhta hai aur billing require karta hai.
Sahi Tarika: Always select `"Create API key in new project"`!

Step-by-Step Key Generation (Sirf 30 Seconds):

1. Browser me **https://aistudio.google.com/api-keys** open karo.
2. Blue button click karo: **"Create API key"**.
3. Modal me select karo: **"Create API key in new project"** (Kisi purane project ko select mat karna).
4. Key generate hogi (ye key `AIzaSy...` se start hoti hai). Click **"Copy key"**.
5. Apne project ki `.env` file me paste karo:

```
# .env File
GEMINI_API_KEY=AIzaSyD-YourCopiedKeyHere1234567890
```

Part 2: Official Google GenAI SDK Setup & Live Test

Google ka official high-performance SDK `@google/genai` install karo:

```
npm install @google/genai dotenv
```

Verification Script (scratch/test-gemini.js):

```
require("dotenv").config();
const { GoogleGenAI } = require("@google/genai");

async function testAI() {
    try {
        const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
        const response = await ai.models.generateContent({
            model: "gemini-2.5-flash",
            contents: "Hello! Reply with: 'Gemini AI Integration Successful!'"
        });
        console.log("AI Response:", response.text);
    } catch (err) {
        console.error("Error:", err.message);
    }
}
testAI();
```

```
✓ Expected Output:
AI Response: Gemini AI Integration Successful!
```

Part 3: Production Architecture — Backend Me AI Kaise Kaam Karta Hai?

1. **Zero Direct Database Mutation:** LLM ko direct database write permission nahi di jaati (hallucination risk). LLM sirf read/analyze karta hai.
2. **Injected Context (Grounding):** LLM ko user ka actual financial balance aur transaction history context me provide ki jaati hai taaki wo fake numbers na banaye.
3. **Redis Caching Layer:** Har baar LLM hit karne par tokens waste hote hain aur 1-2 second lagta hai. Redis Cache se **<5ms** me instant response do!
4. **Redis Rate Limiter:** 10 requests / 60 seconds ka limit lagao taaki koi bot API key exhaust na kare.

```
[ Client / Postman (Port 3000) ]
               |
               ▼
      [ Nginx API Gateway ]
               | (Forward to /api/ai/*)
               ▼
[ Express Router (JWT Auth + Redis Rate Limiter) ]
               |
               ├──> 1. Check Redis Cache (`ai:insights:userId`)? ──[HIT]──> Return in 3ms!
               |
          [CACHE MISS]
               |
               ├──> 2. Fetch User Balances & Ledger Transactions from MongoDB
               |
               ├──> 3. Call Gemini 2.5 Flash with Grounded Prompt + JSON Schema
               |
               ├──> 4. Save Response in Redis with 10-Minute TTL (600s)
               |
               ▼
        [ Return Clean Structured JSON to Client ]
```

Part 4: AI Service Implementation (src/services/ai.service.js)

```
// src/services/ai.service.js
const { GoogleGenAI } = require("@google/genai");

const getGeminiClient = () => {
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) throw new Error("GEMINI_API_KEY not defined in .env");
    return new GoogleGenAI({ apiKey });
};

// 1. AI Financial Advisor (Structured JSON Output)
const generateSpendingInsights = async (financialContext) => {
    const ai = getGeminiClient();

    const prompt = `
You are VaultX AI Financial Advisor for an advanced banking system.
Analyze user financial data:
- Accounts: ${financialContext.accountsCount} | Balances: ${JSON.stringify(financialContext.balance
s)}
- Recent Transactions: ${JSON.stringify(financialContext.transactions, null, 2)}

Instructions:
1. Determine financial health (EXCELLENT, GOOD, MODERATE, ATTENTION_NEEDED).
2. Summarize spending habits.
3. Provide 3 crisp actionable tips to increase savings.
4. Return ONLY valid JSON:
{
  "financialHealth": "EXCELLENT | GOOD | MODERATE | ATTENTION_NEEDED",
  "summary": "Brief 2-3 sentence overview",
  "totalInflow": 0, "totalOutflow": 0,
  "keyObservations": ["point 1", "point 2"],
  "actionableTips": ["tip 1", "tip 2", "tip 3"]
```

```
}
`;

    const response = await ai.models.generateContent({
        model: "gemini-2.5-flash",
        contents: prompt,
        config: { responseMimeType: "application/json" } // 100% strict JSON schema
    });

    return JSON.parse(response.text);
};

// 2. Natural Language Banking Chat Assistant
const chatWithFinancialAssistant = async (userMessage, financialContext) => {
    const ai = getGeminiClient();
    const systemPrompt = `
You are VaultX Banking Assistant. Answer based ONLY on user's real financial data:
${JSON.stringify(financialContext, null, 2)}
Rules: Answer in friendly Hinglish/English. Use ONLY given numbers. Never invent balances.
`;

    const response = await ai.models.generateContent({
        model: "gemini-2.5-flash",
        contents: [{ role: "user", parts: [{ text: `${systemPrompt}\n\nUser: ${userMessage}` }] }]
    });

    return response.text;
};

module.exports = { generateSpendingInsights, chatWithFinancialAssistant };
```

Part 5: Controller with Redis Caching (src/controllers/ai.controller.js)

```
// src/controllers/ai.controller.js
const accountModel = require("../models/account.model");
const transactionModel = require("../models/transaction.model");
const aiService = require("../services/ai.service");
const redisClient = require("../config/redis");

// Context Aggregator
const getUserFinancialContext = async (userId) => {
    const accounts = await accountModel.find({ user: userId });
    const accountIds = accounts.map(a => a._id);
    const balances = [];

    for (const acc of accounts) {
        balances.push({
            accountId: acc._id.toString(),
            status: acc.status,
            balance: await acc.getBalance()
        });
    }

    const rawTx = await transactionModel.find({
        $or: [{ fromAccount: { $in: accountIds } }, { toAccount: { $in: accountIds } }]
    }).sort({ createdAt: -1 }).limit(10).lean();

    const transactions = rawTx.map(tx => ({
        type: accountIds.some(id => id.toString() === tx.fromAccount?.toString()) ? "DEBIT" : "CREDI
T",
        amount: tx.amount,
        status: tx.status,
        date: tx.createdAt
    }));

    return { accountsCount: accounts.length, balances, transactions };
};
```

```
// 1. GET /api/ai/insights (With Redis 10-Min Cache)
const getSpendingInsightsController = async (req, res) => {
    try {
        const userId = req.user._id.toString();
        const cacheKey = `ai:insights:${userId}`;

        // Step 1: Redis Cache Check
        const cached = await redisClient.get(cacheKey);
        if (cached) {
            return res.status(200).json({ success: true, source: "redis_cache", data: JSON.parse(cach
ed) });
        }

        // Step 2: Fetch Context & Call AI
        const context = await getUserFinancialContext(userId);
        if (context.accountsCount === 0) {
            return res.status(400).json({ success: false, message: "No bank accounts found." });
        }

        const insights = await aiService.generateSpendingInsights(context);

        // Step 3: Cache in Redis with 10-Minute TTL (600s)
        await redisClient.set(cacheKey, JSON.stringify(insights), "EX", 600);

        return res.status(200).json({ success: true, source: "gemini_ai", data: insights });
    } catch (error) {
        return res.status(500).json({ success: false, message: error.message });
    }
};

// 2. POST /api/ai/chat
const chatWithAiController = async (req, res) => {
    try {
        const userId = req.user._id.toString();
        const { message } = req.body;
        if (!message) return res.status(400).json({ message: "message string is required" });

        const context = await getUserFinancialContext(userId);
        const reply = await aiService.chatWithFinancialAssistant(message, context);
        return res.status(200).json({ success: true, reply });
    } catch (error) {
        return res.status(500).json({ success: false, message: error.message });
    }
};

module.exports = { getSpendingInsightsController, chatWithAiController };
```

Part 6: Routes & Nginx API Gateway Setup

```
// src/routes/ai.routes.js
const express = require("express");
const authMiddleware = require("../middleware/auth.middleware");
const aiController = require("../controllers/ai.controller");
const { createRateLimiter } = require("../middleware/rateLimit.middleware");

const router = express.Router();
// 10 calls / 60 seconds per user
const aiLimiter = createRateLimiter({ prefix: "ai", windowSeconds: 60, maxRequests: 10 });

router.get("/insights", authMiddleware.authMiddleware, aiLimiter, aiController.getSpendingInsightsCon
troller);
router.post("/chat", authMiddleware.authMiddleware, aiLimiter, aiController.chatWithAiController);

module.exports = router;
```

Part 7: Live Testing Guide & Verification Outputs

Test 1: Spending Insights API (First Call -> Gemini AI Call)

```
curl -H "Authorization: Bearer YOUR_JWT_TOKEN" http://localhost:3000/api/ai/insights
```


```
{
  "success": true,
  "source": "gemini_ai",
  "data": {
    "financialHealth": "GOOD",
    "summary": "Your account shows positive net cash flow with healthy initial deposits.",
    "totalInflow": 50000,
    "totalOutflow": 8200,
    "keyObservations": [
      "Consistent initial deposits received",
      "Low transaction volume indicates healthy reserve"
    ],
    "actionableTips": [
      "Maintain at least 30% of deposits in your savings reserve.",
      "Track non-essential expenses using automated monthly summaries.",
      "Consider setting up recurring auto-savings for emergency funds."
    ]
  }
}
```


Test 2: Spending Insights API (Second Call -> Sub-5ms Redis Cache Hit)

Same endpoint dusri baar hit karne par Gemini call nahi hoti, seedha Redis Cache se instant response aata hai:


```
{
  "success": true,
  "source": "redis_cache",   <-- INSTANT 3ms RESPONSE!
  "data": { ... }
}
```


Test 3: Natural Language AI Banking Assistant

```
curl -X POST http://localhost:3000/api/ai/chat \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"message": "Bhai mera account balance kitna hai aur kya maine koi spend kiya?"}'
```


```
{
  "success": true,
  "reply": "Namaste! Aapke VaultX Savings Account me abhi kul ₹41,800 ka balance uplabdh hai. Pichle records ke mutabiq aapne haal hi me ₹8,200 ka debit kiya tha. Agar aapko koi naya transfer karna hai, toh please secure /api/transactions endpoint ka use karein!"
}
```

Part 8: Top 5 Production Errors & Instant Fixes

Error / Symptom	Kyun Aata Hai?	Instant 1-Minute Solution
`403 PERMISSION_DENIED (free tier not available)`	API key kisi purane Google Cloud project me generate hui thi jisme billing required hai.	**Fix:** AI Studio me jao, click "Create API key", aur select karo **"Create API key in new project"**. Free key mil jayegi.
`429 ResourceExhausted (Quota exceeded)`	15 requests/minute ki free rate-limit cross ho gayi.	**Fix:** Redis Caching layer lagao (10-min TTL) aur Redis sliding window rate-limiter lagao taaki same requests repeat na hon.
`SyntaxError: Unexpected token in JSON`	LLM ne JSON ke aage-peeche markdown backticks ```` ```json ```` laga diye.	**Fix:** `config: { responseMimeType: "application/json" }` use karo. Isse Gemini 100% strict valid JSON string return karta hai.
`Hallucinated Balance (Galat balance batana)`	LLM ko prompt me user ka actual financial context nahi diya gaya.	**Fix:** System instruction me `account.getBalance()` inject karo aur strict rule do: "*Use ONLY provided numbers*".
`Server Crash on Gemini API Error`	Uncaught exception in async call.	**Fix:** Controller me `try / catch` block lagao aur error hone par clean JSON error response do.

Interview Ready Pitch:
"Maine Google Gemini 2.5 Flash SDK integrate kiya hai with Structured JSON Outputs. Production scalability ke liye maine Redis Caching layer lagayi hai jo repeated queries par LLM calls bypass karke sub-5ms responses deti hai, aur Redis Sliding Window Rate-Limiter lagaya hai jo API abuse prevent karta hai. Data integrity ke liye LLM kabhi direct database write nahi karta; wo purely analytical insights provide karta hai."